

REUSABLE COMPLEXITY & MATH UTILITIES

Common Asymptotic Computation Helpers:

```
// 1. Log2 Computation: log2(N) = log(N) / log(2)
int log2(int n) { return (int)(Math.log(n) / Math.log(2)); }

// 2. Power of 2 Check: n & (n - 1) == 0 (for n > 0)
boolean isPowerOfTwo(int n) { return n > 0 && (n & (n - 1)) == 0; }

// 3. Fast Bitwise Shift for 2^N: (1 << N)
long powerOfTwo(int n) { return 1L << n; }
```

💡 AHA MOMENT #1: BIG-O IS WORST-CASE UPPER BOUND ON GROWTH
Big-O $O(f(N))$ measures how execution time or memory space grows as the input size N approaches infinity. It ignores hardware speed, compiler optimizations, and constant factors to isolate algorithmic efficiency!

1. BIG-O (O)

Worst-Case Upper Bound
Guarantees algorithm will take NO MORE THAN $f(N)$ time.
Used 99% of time in FAANG interviews.

2. BIG-OMEGA (Ω)

Best-Case Lower Bound
Guarantees algorithm will take AT LEAST $f(N)$ time.
E.g., Linear scan best case is $\Omega(1)$.

3. BIG-THETA (Θ)

Tight Bound
Algorithm is BOTH $O(f(N))$ AND $\Omega(f(N))$.
Exact growth rate.
MergeSort is $\Theta(N \log N)$ in all cases.

THE 3 SIMPLIFICATION RULES

- Rule 1: Drop Constants** — $O(2N) \rightarrow O(N)$. Two sequential loops over N is still linear $O(N)$.
- Rule 2: Drop Non-Dominant Terms** — $O(N^2 + N + 100) \rightarrow O(N^2)$. As $N \rightarrow \infty$, N^2 completely dominates N .
- Rule 3: Additive vs Multiplicative** — Sequential steps add ($O(A + B)$), nested loops multiply ($O(A \times B)$).

 ASYMPTOTIC GROWTH RATE RANKING MATRIX

Notation	Name	N = 10	N = 1,000	Max N For 1 Sec	Example Algorithms
$O(1)$	Constant	1	1	10^9+	Array index lookup, HashMap get avg, Stack push
$O(\log N)$	Logarithmic	3	10	10^{18}	Binary search, Balanced BST lookup, Euclidean GCD
$O(N)$	Linear	10	1,000	10^7	Single loop scan, Two Pointers, Sliding Window
$O(N \log N)$	Linearithmic	33	10,000	10^6	MergeSort, QuickSort avg, HeapSort, PriorityQueue scan
$O(N^2)$	Quadratic	100	10^6	10,000	Nested loops, Bubble Sort, 2D Grid DP
$O(2^N)$	Exponential	1,024	10^{301} (Fails)	20 - 25	Backtracking subsets, Fibonacci naive recursion
$O(N!)$	Factorial	3.6×10^6	∞ (Fails)	10 - 12	Permutations generation, Travelling Salesperson

⚡ **FAANG BENCHMARK:** Modern CPUs execute roughly 10^8 operations per second. Keep total operations under 10^8 to pass online judge time limits!

EXAMPLE 1: NESTED VS INDEPENDENT LOOPS

```
// Independent Loops: O(A + B)
for (int a : arrayA) { ... } // O(A)
for (int b : arrayB) { ... } // O(B)

// Nested Loops: O(A * B)
for (int a : arrayA) {
    for (int b : arrayB) { ... } // O(A * B)
}
```

EXAMPLE 2: LOGARITHMIC LOOP

```
// Loop variable doubles each iteration
for (int i = 1; i < n; i *= 2) {
    // Executes log2(N) times → O(log N)
}

// Loop variable halved each iteration
while (n > 0) { n /= 2; } // O(log N)
```

EXAMPLE 3: AMORTIZED ANALYSIS (ARRAYLIST GROWTH)

When an `ArrayList` reaches capacity, it doubles its array size ($O(N)$ copy). However, doubling happens only at powers of 2 (1, 2, 4, 8, 16, ...).

Total cost for N inserts: $N + 1 + 2 + 4 + \dots + N = N + 2N = 3N \rightarrow O(3N)$.

Dividing by N items: $\frac{3N}{N} = 3 \rightarrow$ **$O(1)$ Amortized Time per insert!**

AUXILIARY SPACE VS TOTAL SPACE

- **Total Space:** Memory used by input + auxiliary memory.
- **Auxiliary Space:** Extra memory allocated by algorithm (excluding input data).

FAANG interviewers care about Auxiliary Space. If input is `int[] arr` of size N , auxiliary space is $O(1)$ unless you allocate a new data structure.

RECURSIVE CALL STACK MEMORY

Every recursive call pushes a frame onto the JVM Call Stack (local variables + return address).

Stack depth = max simultaneous active recursive calls.

Tree DFS call stack space = $O(H)$ where H is tree height (worst case $O(N)$ for skewed tree, $O(\log N)$ for balanced tree).

CALL STACK TRACE DEMO: RECURSIVE FACTORIAL

```
int fact(int n) {
    if (n ≤ 1) return 1;
    return n * fact(n - 1);
}
// fact(4) → fact(3) → fact(2) → fact(1)
// Max stack depth = 4 → Auxiliary Space  $O(N)$ 
```

⚡ N CONSTRAINT TO EXPECTED TIME COMPLEXITY

N Range in Problem	Expected Complexity	Likely Pattern
$N \leq 10 - 12$	$O(N!)$	Permutations, TSP backtracking
$N \leq 20 - 25$	$O(2^N)$	Subsets, Bitmask DP, Combination Sum
$N \leq 100$	$O(N^4)$ or $O(N^3)$	Floyd-Warshall, 3D DP, 4-sum
$N \leq 1,000$	$O(N^2)$	2D DP, Nested loops, All pairs check
$N \leq 100,000$ (10^5)	$O(N \log N)$ or $O(N)$	Sorting, Heap, Two Pointers, Sliding Window, Prefix Sum
$N \leq 1,000,000$ (10^6)	$O(N)$ or $O(N \log N)$	Single pass array scan, HashMap, Fast I/O
$N \geq 10^9$	$O(\log N)$ or $O(1)$	Binary Search on Answer, Math formula, Bitwise

💡 **FAANG TRICK:** Look at the constraints FIRST! If $N = 10^5$, immediately reject $O(N^2)$ brute force ideas and think Sorting / Two Pointers / HashMap / Sliding Window!

1 REVISION CHECKLIST

- ✔ Know the growth order: $O(1) < O(\log N) < O(N) < O(N \log N) < O(N^2) < O(2^N) < O(N!)$
- ✔ Drop constants and non-dominant terms during analysis
- ✔ Always analyze BOTH Time Complexity and Auxiliary Space Complexity
- ✔ Factor in recursive stack depth for space complexity
- ✔ Match N constraint to target time complexity in the first 30 seconds of an interview



- ✗ **Confusing String Concatenation:** `s += char` in a loop over length N takes $O(N^2)$ total time because strings are immutable in Java! Use `StringBuilder` for $O(N)$.
- ✗ **Ignoring String Comparison:** `stringA.equals(stringB)` is $O(L)$ where L is string length, not $O(1)$!
- ✗ **Assuming HashMap `get()` is Always $O(1)$:** Hash collision worst-case is $O(\log N)$ in Java 8+ (Treeified) or $O(N)$ in Java 7.



Script 1 (Time Complexity): "The time complexity is $O(N \log N)$ because we first sort the array of N elements using Dual-Pivot QuickSort, and then perform a single linear scan of $O(N)$."

Script 2 (Space Complexity): "The auxiliary space complexity is $O(K)$ because our Min-Heap holds at most K elements at any time. We do not count the input array."

🚀 Next Step: Move to **Topic 02: Arrays, Strings & Hashing Masterclass**.